CO 2 Assessment - 2

Name : E. Gowtham

Reg. NO : 192311190

Course : Theory of Computation

Code : CSA 1301

1) Design the regular expression for the language Represented by the given automation

step 1: Identify the states and transitions

- Initial state : q_0
- Intermediate state : q_1
- Final state : q_2

The transitions shown in the diagram are:

- $q_0 \rightarrow q_1$ on input 0 or 1
- $q_1 \rightarrow q_2$ on input 0 or 1
- $q_2 \rightarrow q_1$ on input 0 or 1

Step 2: Find the pattern

To reach the final state q_2, the automation must first read one symbol to move from q_0 to q_1; and then another symbol to Reach q_2.

After reaching q_2, it can read two more symbols by moving from q_2 to q_1, and back to q_2.

Therefore, the automation accepts strings of even length, with a minimum length of 2, over the alphabet $\{0,1\}$

Step 3: write the Regular expression

Since, each transition accepts either 0 or 1, we write

$$(0+1)(0+1)$$

The automation can Repeat a pair of symbols any number of times. Therefore, the complete Regular expression is:

$$\boxed{(0+1)(0+1)((0+1)(0+1))^*}$$

Step 4 : Verify with Examples

Input string	Result
00	Accepted
01	Accepted
10	Accepted
11	Accepted
0000	Accepted
0101	Rejected
0	Rejected
101	Rejected
ϵ	

Final answer:

$$(0+1)(0+1)((0+1)(0+1))^*$$

This Regular expression Represents all Binary ~~settings~~ strings of even length greater than or equal to 2, as accepted by the given automation

2) Consider the following grammar

$$S \to aSc \mid T \mid U \mid \epsilon$$

$$T \to bTc \mid \epsilon$$

$$U \to aUb \mid \epsilon$$

A] What is the language defined by grammar

The language generated by the grammar is the union of the languages generated by S, T and U.

- $S \to aSc$ adds one a to the beginning and one c to the end
- $T \to bTc$ generates strings containing equal numbers of b is followed by equal numbers of cs.
- $U \to aUb$ generates strings containing equal numbers of a S followed by equal numbers of b s.
- ϵ represents the empty string.

The languages generated by T and U are:

$$L(T) = \{b^n c^n \mid n \geq 0\}$$

$$L(U) = \{a^n b^n \mid n \geq 0\}$$

The language generated by S is:

$$L(S) = \{a^n x c^n \mid x \in L(T) \cup L(U) \cup \{\epsilon\}, n \geq 0\}$$

Thus, the grammar generates strings formed by wrapping a string from T, U or the empty string with matching numbers of a s and c s.

B] Give the derivation tree for the word aaaabbc.

The word aaabc can be derived as follows:

$$S \Rightarrow aSc$$
$$\Rightarrow aaScc$$
$$\Rightarrow aaUcc$$
$$\Rightarrow aaaUbcc$$
$$\Rightarrow aaabbcc$$

This derives aaabbcc, not aaabbc. Therefore, the Requested word aaabbc cannot be generated by the given grammar.

C) Is the above grammar ambiguous? justify

Yes, The grammar is ambiguous.

A grammar is ambiguous. if at least one string has two or more distinct derivations or parse trees.

for the empty string ϵ we have three different derivations

Derivation 1:

$$S \Rightarrow \epsilon$$

Derivation 2:

$$S \Rightarrow T \Rightarrow \epsilon$$

Derivation 3:

$$S \Rightarrow U \Rightarrow \epsilon$$

The same string ϵ can be generated in three different ways.

Conclusion:

The grammar is ambiguous because the empty string has multiple distinct derivations.

3) Design an NFA with ϵ- transitions for the Regular expression $00^*1 + 11^*0$.

The given regular expression is:

$$00^*1 + 11^*0$$

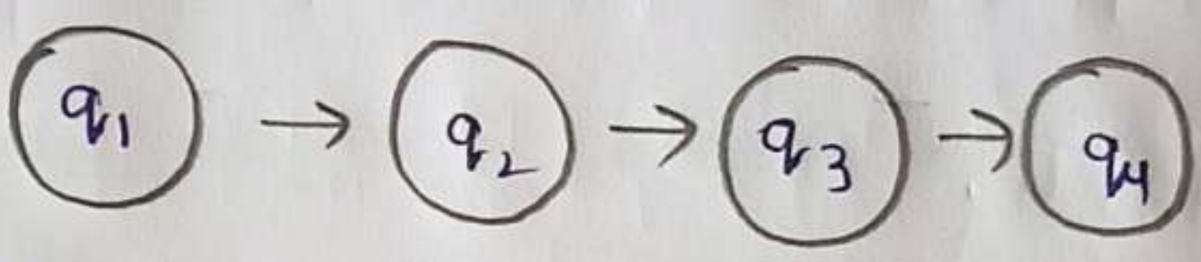
Here, + Represents the union (OR) operation.

The expression contains two parts:

- 00^*1 : one or more zeroes followed by one 1.
- 11^*0 : one or more ones followed by one 0.

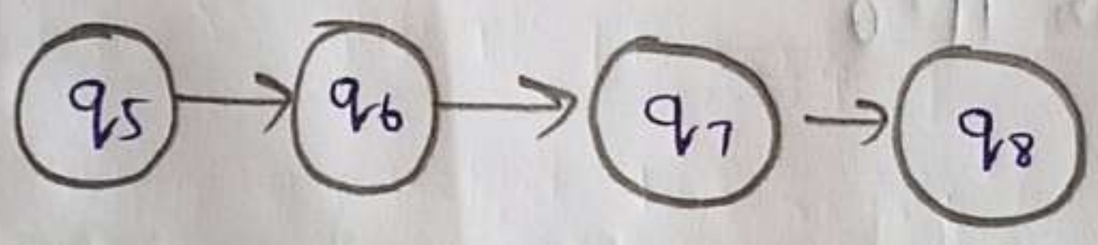
NFA Construction

Branch 1 : 00^*1



Transitions: $q_1 \to 0 \to q_2$; q_2 loops on 0; $q_2 \to 1 \to q_4$ (final)

Branch 2: 11^*0



Transitions: $q_5 \to 1 \to q_6$: q_6 loops on 1; $q_6 \to 0 \to q_8$ (final).

To combine both branches, introduce a new initial state q_0, with ϵ- transitions to q_1 and q_5.

- Initial state : q_0
- ϵ-transitions : $q_0 \to q_1$, $q_0 \to q_5$
- final states : q_4 and q_8.

Transition Table

state	Input 0	Input 1	Input ε
→q_0	–	–	q_1, q_5
q_1	q_2	–	–
q_2	q_2	q_4	–
*q_4	–	–	–
q_5	–	q_6	–
q_6	q_8	q_6	–
*q_8	–	–	–

Conclusion:

The ε- NFA accepts strings matching either

00^*1 or 11^*0

4) i) Show that if L_1 and L_2 are regular, then $L_1 + L_2$ is also Regular.

Let L_1 and L_2 be two Regular languages,

Since they are Regular, there exist finite automata M_1 and M_2 that accept them.

we can construct a new NFA by introducing a new initial state q_0 and adding ε-transitions from q_0 to the initial states of M_1 and M_2

The new automaton accepts a string if it is accepted by either M_1 or M_2.

Therefore, the language accepted by the new automaton is:

$$L = L_1 \cup L_2$$

Since an NFA with ε-transitions can be converted into an equivalent DFA, the Resulting language is Regular.

Conclusion:

Regular languages are closed under union. Hence, if L_1 and L_2 are Regular, $L_1 \cup L_2$ is also regular.

ii) is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify

No, the language is not regular.

Let

$$L = \{a^p \mid p \text{ is prime}\}$$

Assume that L is regular. By the pumping lemma, there exists a pumping length n.

Choose a prime number $p > n$ and the string:

$$w = a^p$$

According to the pumping lemma, $w = xyz$, where:

- $|xy| \leq n$
- $|y| > 0$
- $xy^i z \in L$ for every $i \geq 0$

since the string contains only a s, let $y = a^k$, where $k \geq 1$.

choose $i = p+1$, the Resulting string has length:

$$|xy^{p+1}z| = p + pk = p(1+k)$$

Because p is Prime and $k \geq 1$, the number $p(1+k)$ is composite.

Therefore, $xy^{p+1}z \notin L$, contradicting the pumping lemma,

Conclusion:

The language $\{a^p \mid p \text{ is prime}\}$ is not Regular

5) Design a minimized automaton equivalent to the given DFA.

The given DFA containg six states:

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$

From the diagram:

- Initial state : q_0
- Final state : q_1, q_2, q_4
- Non-Final states: q_0, q_3, q_5

Step 1: Identify equivalent states

Two states are equivalent. if they accept the same set of Future input strings.

From the given transition diagram:

- q_0 and q_3 are equivalent because both are non-final transition to each other on input 0 and Transition to equivalent final states on input 1.
- q_1 and q_4 are equivalent because both are final, Transition to q_2 on input 0, and transition to q_5 on input 1.

The states q_2 and q_5 are distinguishable because q_2 is final while while q_5 is a non-final dead state.

Therefore, the equivalent state groups are:

$$A = \{q_0, q_3\}$$

$$B = \{q_1, q_4\}$$

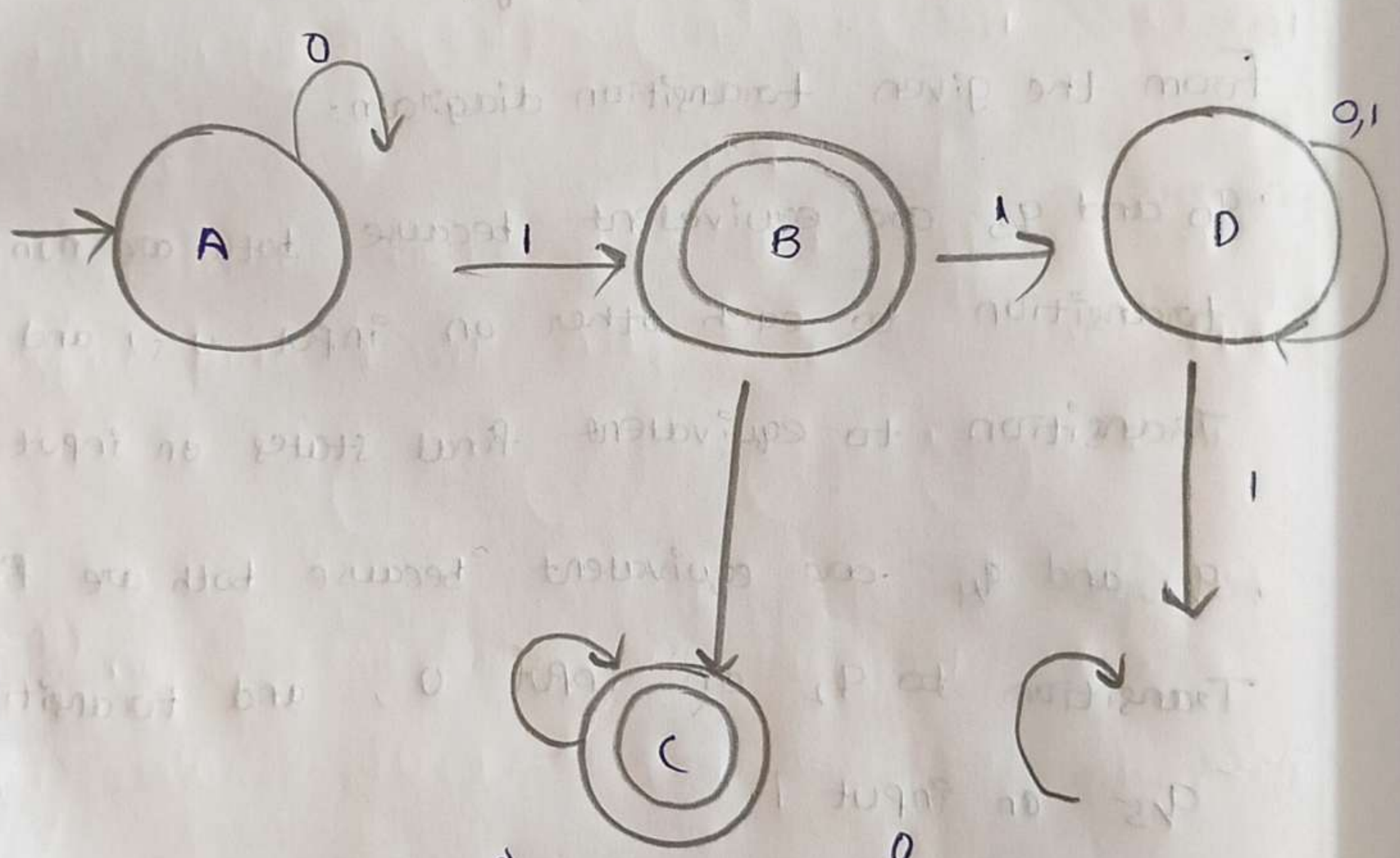
$$C = \{q_2\}$$

Step 2: Construct the minimized transition table

state	input 0	input 1
→A	A	B
*B	C	D
*C	C	D
D	D	D

Here, → denotes the intial state and * denotes a final state.

Step 3: Draw the minimized DFA



Step 4: Result

The minimized DFA has four states:

- $A = \{q_0, q_3\}$
- $B = \{q_1, q_4\}$
- $C = \{q_2\}$
- $D = \{q_5\}$

The intial state is A, and the final states are B and C.